Speaking Notes — Chip Convergence, ICLAD-DAC 2026

Deck: `combined_deck.pdf` · **18 slides** + 5 backup
Budget: ~17 min talk, ~3 min Q&A. Content slides ~1 min; the three FLOW slides (3, 9, 15) are ~45 sec each — you narrate the boxes left to right.
How to use: the *Say* blocks are written to be spoken. Read them or paraphrase. The *Flow* blocks explain the machinery — use them if the room is technical or if someone asks. The ⚠️ blocks are things NOT to say.

If you are running late: cut slide 11 and slide 14 first. Protect 5, 16, 17. The flow slides (3, 9, 15) are quick — do not cut them, they are what you asked to talk to.

Slide 1 — One thesis, three problems · 90 sec

Say:

> Good morning. I'm Harikrishnan, competing solo as Chip Convergence, and I entered all three tracks — NVIDIA RTL optimization, NXP SoC generation, and ASU layout repair.
>
> They look like three different problems. They're really one problem.
>
> A language model will hand you hardware that is smaller, faster, and *wrong*. And plausible-but-wrong RTL is worse than no RTL at all — because it passes review, and then it fails in silicon.
>
> So we didn't build an agent and then add checks. We built the verifier first, and put the model behind it. The model proposes. Deterministic gates dispose. And the system's
> default is to ship nothing.

Slide 2 — NVIDIA: improve PPA, stay functionally identical · 70 sec

Say:

> The NVIDIA problem: seven IPs, their testbenches, a Yosys and OpenSTA flow on a 7nm
> library. Rewrite the RTL to improve area and timing — without changing what it does.
>
> Every candidate goes through five layers. Lint. Compile. The IP's own testbench. Then formal equivalence — Yosys has to *prove* the candidate equivalent to the original. Then cycle-exact co-simulation of both designs side by side.
>
> The word "proven" is doing real work there. It binds five conditions at once — the tool exited clean, it printed success, it actually compared something, everything it compared was proven, and nothing was left unproven. Anything short of all five is inconclusive, and inconclusive is never a pass.
>
> And the selection rule: to be shippable, a candidate must be formally proven. A better

Slide 3 — NVIDIA: the flow, end to end · 45 sec

Say:

> Here's the whole loop, left to right.
>
> **Diagnose** — we synthesize, run static timing, and attribute the critical path back to a specific source file. Then we classify what that path is *made of* from the cell mix. Zero tokens so far.
>
> **Select** — the structure tag picks which transforms are applicable, and a risk gate based on how badly timing is failing decides which are affordable.
>
> **Propose** — that's the only model call. And notice what it's being asked: one named transform, on one named cone, with the guard conditions. Never "make this faster."
>
> **Verify** — five layers, and the model gets no vote here.
>
> **Decide** — the selector admits only formally-proven candidates. If none qualify, it

Slide 4 — Results, and the ones we refused · 75 sec

Say:

> The results. prim at 0.58 — that's area-delay product against the baseline, so lower is better. Nearly a 42% improvement, formally proven, with power down 70%. sha512 at
> 0.727, full five layers, and it moved timing from failing by 97 picoseconds to passing with 335 to spare.
>
> But I want to spend the time on the two we *refused*.
>
> On ascon, a candidate beat our banked result. Our diagnostic found a real counterexample — actual inputs where it computes something different. Refused.
>
> And on sha512, during a live run, a candidate came in at 0.65 — dramatically better than the 0.727 we ship. It wasn't formally proven. The system refused it and kept the weaker result.

Slide 5 — The 4% win that was the bug · 100 sec ⭐ SLOW DOWN HERE

Say:

> Our agent found a 4% improvement on an asynchronous FIFO.
>
> It passed lint. It passed compile. It passed the testbench. Formal equivalence **proved** it. Cycle-exact differential simulation passed. Five for five — it was the best-verified candidate we had produced.
>
> *(pause)*
>
> It was broken.
>
> It had removed the registers on the Gray-code pointer and computed the encoding combinationally instead. As a *function*, those are identical — which is exactly why formal proved it. But that pointer crosses a clock domain boundary. A combinational

Slide 6 — NVDLA: whole-design formal at scale · 70 sec

Say:

> NVDLA is the big one — around 950,000 cells. We proved whole-design equivalence across
> 381,209 equivalence points through our production recipe. That is uncommon at this scale.
>
> Our release packet is six of six. Every gate control demonstrated live — including the negatives, which matter more: a deliberately mutated design must FAIL, and it does.
>
> Here's the part I'd rather tell you than have you find. That packet sat at five of six for a day. The failing check turned out to be a bug in *our own log parser* — it was counting the word "failed" inside the test runner's help text as a test failure. We fixed it, re-ran, and got six of six. And critically, the negative control still fires, which proves we fixed the parser rather than blinding the gate.

Slide 7 — NXP: generate a secure SoC from a diagram · 70 sec

Say:

> Different problem. Here you're given an architecture document and a testbench port skeleton, and you have to produce synthesizable RTL for a whole SoC.
>
> Notice what changes: there's no reference design. Nothing to prove equivalence *against* — you're generating something new. So the proof technology has to change.
>
> Instead of formal equivalence, we built a correctness firewall: contracts and independent oracles.

Flow:

> The model reads the diagram and emits a YAML spec per IP block. Those specs are validated *before* a single line of Verilog exists — a bad spec costs one re-prompt instead of a full synthesis cycle. Then the organizer's generator library builds the

Slide 8 — The night the hidden problems landed · 75 sec

Say:

> Two nights ago the organizers released two new problems — a network-on-chip AES crypto SoC, and a multi-domain crypto SoC with four AES engines and a 4-by-3 mesh.
> Architectures our agent had never seen.
>
> We ran them. And they exposed nine defects — in *our* agent, not the model.
>
> Three worth naming. We had hardcoded the top module name to the first problem, so every
> other problem failed to elaborate. Our prompt budget was sized for eight IP blocks, so on a 22-block design fourteen of them were silently truncated away — the model was guessing ports it had never been shown. And our port parser was reading a module's *parameter* list as its port list, which meant any parameterized module reported zero

Slide 9 — NXP: the flow, end to end · 45 sec

Say:

> Same shape, different proof technology.
>
> **Read** — we parse the architecture document, and we derive the top module's name from
> the organizer's own testbench skeleton rather than assuming it. That one detail is why the two new problems work at all.
>
> **Spec** — the model emits a YAML spec per IP block, and those are validated *before a single line of Verilog exists*. A bad spec costs one re-prompt instead of a synthesis cycle.
>
> **Generate** — the organizers' own library builds the RTL from those specs. Deterministic.

Slide 10 — Instructions are probabilistic. Gates are not. · 80 sec ⭐

Say:

> This is the experiment I'd most like you to take away.
>
> The model was tying constant values to *output* ports — which is illegal Verilog, and kills elaboration.
>
> Step one: we checked whether it had the information. It did. The port directions were right there in the prompt — "output wire, one-bit, this name." It ignored them.
>
> Step two: we added an explicit rule. "An output port may never be connected to a constant." It complied on the first problem — two runs out of two. And violated it on the second.
>
> Step three: we added a deterministic gate that compares every connection against the

Slide 11 — What we do and do not claim · 50 sec *(cut this first if late)*

Say:

> Briefly, the boundaries. We claim: two calls, thirty out of thirty, 79 out of 79 on both oracles for the released problem. And for the two new ones — RTL that conforms to the
> port contract and elaborates, on architectures we'd never seen.
>
> We do *not* claim those are functionally correct. No golden testbench ships for any tier, including the original — so they cannot be scored. "Elaborates and conforms" is not "correct," and I'd rather say that than have you assume otherwise.

Slide 12 — ASU: repair DRC violations · 65 sec

Say:

> Third track. You're given a layout script with design-rule violations, and you have to repair them. Scored on final violation rate — lower is better — gated on the repair being
> valid and preserving connectivity.
>
> Here's the result that I think shows the most judgment: we took the model *out*.
>
> The winning repair is deterministic. Each flagged multi-cut via array gets replaced with
> one continuous via bar — and that's derived directly from the design rule itself, not fitted to the examples.
>
> Partly that was forced. The official agent image is a bare Python container with no KLayout binary, mounted read-only. Our development agent measures every

Slide 13 — v2 Rev3: all seven, electrically proven · 90 sec ⭐

Say:

> This morning we reopened this track, and the story got much better — in both directions.
>
> First, the good news: we found a second seeded pattern. Whole routing tracks had been
> translated off the grid. The inverse transform — translate them back, carrying their vias
> along — cleared the biggest remaining violation class.
>
> Then the uncomfortable news: an independent layer-aware review of our OWN submitted
> repair found that some of those via bars electrically joined nets that should be
> separate. Forty-nine shorts, across four blocks — invisible to the official connectivity

Slide 14 — Why it generalized · 50 sec *(cut second if late)*

Say:

> No tuning. No retraining. No code change. The blocks were released and the agent scored
> them in the same band.
>
> That's not luck — it's a consequence of the design. A model optimized against five public
> blocks has no particular reason to transfer. A transform decoded from the design rule itself has every reason to.
>
> Two honesty notes we put on the slide before anyone had to ask. Doing nothing is *not* a
> score of 1.0 — the ratio is computed differently, and it's about 1.25. And this is a net win, not a "can never be worse" transform: some individual violation classes appear

Slide 15 — ASU: the flow, and why the model left it · 45 sec

Say:

> This one has two agents, and the difference is the point.
>
> **Diagnose** — we classify each DRC violation and match it against our repair-rule library.
>
> **Repair** — the via-bar transform. Fully deterministic, derived from the rule.
>
> **Measure** — and this is the important box: our development agent measures every candidate
> with the *organizers' own evaluator*, in the version-pinned KLayout. Our scorer is their scorer.
>
> **Ship** — but the agent we actually submit is seven functions, stdlib only, and makes **zero** model calls.

Slide 16 — Three domains, one finding · 60 sec ⭐ THE SYNTHESIS

Say:

> Three tracks. Three different proof technologies. One finding.
>
> On NVIDIA, a candidate passed all five layers including formal proof and was still unsafe — and the answer wasn't more checking, it was a structural rule.
>
> On NXP, the model had the port directions, ignored them; obeyed an explicit rule on one
> problem and broke the other; and only the deterministic gate worked every time.
>
> On ASU, the hidden blocks scored in-band untouched — because the transform was derived
> from the rule rather than fitted to the data.

Slide 17 — What formal and simulation cannot see · 75 sec ⭐ THE CLOSE

Say:

> I want to end on the thing that surprised me most.
>
> Formal equivalence and zero-delay simulation both reason in a model where glitches and
> physical side channels don't exist. So there are two classes of edit that pass every functional check we can run and are still broken.
>
> The first is removing side-channel masking. On a masked AES S-box, un-masking preserves
> the logic function *exactly* — so formal returns "proven" — while destroying the countermeasure. Formally equivalent. Cryptographically broken.
>
> The second is the CDC glitch we saw on slide 4.

Slide 18 — Every claim, one command · 45 sec

Say:

> Everything I've shown you is one command away. Per-artifact assurance is recorded in each
> manifest — not a global claim, a per-result one. The NXP and ASU results reproduce byte-identically. The NVIDIA artifacts re-synthesize exactly.
>
> One distinction I'd flag: the artifacts reproduce; the *search* is stochastic. A shallower run legitimately produced a worse sha512 number than a deeper one. We say so rather than
> quietly reporting the best.
>
> The architecture in one line: the model proposes, deterministic gates dispose, and the default is to ship the baseline.
>
> Thank you — happy to take questions.

Q&A — likely questions

"How do you know the model didn't just break it?"

> Five gates, and the strongest is formal equivalence — nothing becomes shippable without a
> proof. And we have live examples of it firing: an ascon candidate that beat our banked number was refused on a real counterexample.

"Isn't formal equivalence enough? Why the other layers?"

> No — and slide 4 is the proof. Formal answers "same function?", not "same hardware."

"Show me the CDC check in the code."

> The fence mechanism is `ppa/proposer.py`, enforced by `fence_violation`, mandatory for

Backup slides — when to jump

question	go to
"walk me through the NVIDIA flow"	**B1**
"how does NXP generation actually work"	**B2**
"what does the ASU agent do"	**B3**
"what are your actual numbers"	**B4**
"what doesn't work / what are the limits"	**B5**

B5 is your friend. If someone probes for weaknesses, going *to* the slide that lists them is far stronger than defending.